

Auto Flight: Autonomes Segelflugzeug für Flächenabdeckungsflüge

Technische Dokumentation zur Projektaufgabe Embedded Systems

Tom Arlt, 1335730

28.07.2026

Inhaltsverzeichnis

1	Einleitung	1
2	Zielsetzung und Anwendungsszenario	2
2.1	Grundidee	2
2.2	Anwendungsszenario	2
2.3	Bandbreiten- und Datenhaltungskonzept	3
2.4	Start und Landung	3
3	Anforderungen und Randbedingungen	3
3.1	Funktionale Anforderungen	3
3.2	Nicht-funktionale Anforderungen und Randbedingungen	3
3.3	Technische Randbedingungen	3
4	Systemarchitektur	4
4.1	Hardwarearchitektur	4
4.1.1	Hauptkomponenten	4
4.1.2	Mechanischer Aufbau	4
4.1.3	Systemtopologie	4
4.2	Softwarearchitektur	5
4.2.1	Projektstruktur	5
4.2.2	Kommunikationsschichten	5
4.2.3	Web-Stack der Bodenstation	6
4.2.4	Frontend	6
5	Technische Umsetzung und wesentliche Designentscheidungen	6
5.1	Routenplanung	6
5.2	Flugregelung	7
5.3	Sensorik und Datenfusion	7
5.4	Kommunikation Flugcontroller / Motor-Controller	7
5.5	Manuelle Übersteuerung	8
5.6	Wesentliche Designentscheidungen im Projektverlauf	8
5.7	Begleitende Werkzeuge	8
6	Tests und Evaluation	8
6.1	Automatisierte Tests	8
6.2	Evaluation anhand des Entwicklungsverlaufs	9
7	Bekannte Einschränkungen und mögliche Verbesserungen	9
8	Individuelle Beiträge	10

1 Einleitung

„Auto Flight“ ist ein autonomes Segelflugzeug, das vordefinierte Flächen (z. B. landwirtschaftliche Felder) selbstständig in einem Mäandermuster abfliegt. Die Navigation erfolgt über eine Kombination aus GPS, Beschleunigungssensor/Gyroskop (IMU), Magnetometer und Barometer. Das Flugzeug steht während

des gesamten Flugs über eine Langstrecken-Funkverbindung (LoRa) mit einer Bodenstation in Kontakt, die eine Web-Oberfläche zur Missionsplanung und Live-Überwachung bereitstellt. Im Notfall kann das Flugzeug jederzeit über eine handelsübliche RC-Fernsteuerung manuell übernommen werden.

Das System besteht aus drei eigenständigen Firmware-/Software-Projekten (dem Flugcontroller im Flugzeug, der Bodenstation und einem Servo-/Motor-Controller) sowie einer Reihe geteilter Komponenten und Werkzeuge:

Projekt	Plattform	Aufgabe
<code>flight_controller/</code>	ESP-IDF (ESP32-S3)	Hauptrechner im Flugzeug: Routenplanung, Sensorik, Flugregelung, LoRa-Kommunikation
<code>base_station/</code>	ESP-IDF (ESP32-S3)	Bodenstation: WLAN-Zugangspunkt, Web-UI-Server, LoRa-Relais
<code>motor_controller/</code>	PlatformIO/Arduino (ATmega328)	Steuerflächen- /Motoransteuerung, SBUS-Empfang, Sicherheits-Override
<code>frontend_web/</code>	Vue 3 + TypeScript	Bedienoberfläche der Bodenstation (Missionsplanung, Live-Telemetrie)
<code>shared_components/</code>	ESP-IDF-Komponenten	Gemeinsame Bausteine für Flugcontroller und Bodenstation (LoRa, I2C, Sensor-Treiber, Datenhaltung)

Das Projekt ist eine studentische Einzelarbeit im 12. Fachsemester (Modul „Embedded Systems“, Studiengang MRB) und befindet sich im Status eines fortgeschrittenen Prototyps. Ein fortlaufendes Entwicklungsprotokoll mit Fortschritt und aufgetretenen Problemen findet sich in [resources/learning.md](#).

2 Zielsetzung und Anwendungsszenario

2.1 Grundidee

Ausgangspunkt des Projekts (siehe [Projekt_Idee.md](#)) ist ein autonomes Segelflugzeug, das definierte Flächen abfliegt, um sie später, nach einer Erweiterung um eine Kamera, für die Luftbildauswertung landwirtschaftlicher Flächen zu nutzen, etwa zur Ertragsschätzung, Unkrauterkenennung oder allgemeinen Zustandsbeurteilung von Feldern. Die im Rahmen dieser Arbeit umgesetzte Version deckt den navigatorischen Kern dieses Szenarios ab: das zuverlässige, autonome Abfliegen einer vom Nutzer definierten Fläche entlang eines systematisch berechneten Streifenmusters, inklusive Fernüberwachung und -konfiguration sowie eines manuellen Sicherheits-Overrides. Eine Kamera-Nutzlast ist konstruktiv nicht ausgeschlossen, aber (noch) nicht Teil der Implementierung (siehe Abschnitt „Bekannte Einschränkungen“).

2.2 Anwendungsszenario

1. Der Bediener verbindet sich am Boden per WLAN mit der Bodenstation und öffnet die Web-Oberfläche.
2. Über eine Kartenansicht wird das abzufliegende Gebiet als Polygon eingezeichnet und an die Bodenstation übermittelt.
3. Die Bodenstation überträgt das Gebiet per LoRa an das Flugzeug. Der Flugcontroller berechnet daraus eine Mäanderroute mit konfigurierbarem Bahnabstand und meldet die geplante Route zur Vorschau zurück.
4. Nach Freigabe durch den Bediener fliegt das Flugzeug die Route autonom ab. Währenddessen werden Position, Sensordaten und Verbindungsstatus laufend per LoRa übertragen und in der Web-Oberfläche live dargestellt.
5. Der Sicherheitspilot kann jederzeit über einen Schalter am RC-Sender die Kontrolle über die Steuerflächen und den Antrieb direkt übernehmen, unabhängig davon, ob der Flugcontroller funktionsfähig ist.

2.3 Bandbreiten- und Datenhaltungskonzept

Da LoRa bei großer Reichweite nur eine sehr geringe Bandbreite bietet, ist die Funkstrecke bewusst auf kompakte Status- und Steuerdaten (Position, Sensorwerte, Routen, Verbindungsstatus) beschränkt. Größere Nutzdaten wie Kamerabilder sind in der ursprünglichen Idee für eine lokale Speicherung (SD-Karte) oder eine WLAN-Übertragung nach der Landung vorgesehen und bewusst nicht Teil der LoRa-Strecke. Dieser Teil ist im aktuellen Funktionsumfang nicht umgesetzt, da keine Kamera-Nutzlast integriert wurde.

2.4 Start und Landung

Für den Start ist ein Handstart mit anschließendem automatischem Steigflug vorgesehen. Für die Landung ist eine Rückkehr zum Startpunkt mit manueller Landung, Auffangen oder einer sensorgestützten Annäherung geplant. In der aktuellen Umsetzung ist der geschlossene Regelkreis für Höhenhaltung und Navigation vorhanden, eine automatisierte Lande- oder Startlogik jedoch nicht abschließend flugerprobt (siehe „Tests und Evaluation“).

3 Anforderungen und Randbedingungen

3.1 Funktionale Anforderungen

- Autonome Navigation entlang eines aus einem nutzerdefinierten Polygon berechneten Streifenmusters (Wegpunktfolge)
- Bestimmung von Position, Lage (Roll/Pitch), Kurs (Heading) und Höhe während des Flugs aus GPS, IMU, Magnetometer und Barometer
- Fernkonfiguration der Zielfläche und Überwachung des Flugs über eine Weboberfläche, erreichbar über einen von der Bodenstation bereitgestellten WLAN-Zugangspunkt
- Telemetrieübertragung (Position, Sensordaten, Verbindungsstatus, Route) über eine Langstrecken-funkverbindung zwischen Flugzeug und Bodenstation
- Jederzeitige manuelle Übersteuerbarkeit der Steuerflächen und des Antriebs über eine RC-Fernsteuerung, unabhängig vom Zustand des Flugcontrollers

3.2 Nicht-funktionale Anforderungen und Randbedingungen

- **Reichweite vs. Bandbreite:** Die Funkstrecke muss auch über größere Entfernungen (Feldgröße) zuverlässig funktionieren. Das bedingt die Wahl von LoRa (868 MHz, ISM-Band) mit entsprechend geringer nutzbarer Datenrate und macht ein eigenes, sparsames Nachrichtenprotokoll notwendig (siehe Abschnitt „Technische Umsetzung“).
- **Echtzeitfähigkeit:** Die Flugregelung läuft mit einer festen Zykluszeit von 50 ms, um auf Lage- und Kursänderungen zeitnah reagieren zu können.
- **Ressourcenbeschränkung:** Als Segelflugzeug ist Gewicht ein limitierender Faktor für Akkukapazität, Sensorik und Servos. Das begrenzt sowohl die Rechenleistung (Mikrocontroller statt Einplatinencomputer) als auch die Anzahl gleichzeitig betreibbarer I2C-Sensoren an einem gemeinsamen Bus.
- **Ausfallsicherheit:** Die manuelle Übersteuerung darf nicht vom Zustand des Flugcontrollers abhängen, da dieser die eigentliche Ausfallquelle ist, gegen die abgesichert werden soll.
- **Rechtlicher Rahmen:** Im Projektverlauf wurde eine Recherche zu rechtlichen Rahmenbedingungen für autonome Fluggeräte durchgeführt (siehe Woche 2 in [resources/learning.md](#)), die insbesondere die Notwendigkeit einer jederzeit verfügbaren manuellen Kontrolle bestätigt.

3.3 Technische Randbedingungen

- Zwei baugleiche Heltec-WiFi-LoRa-32-V3-Boards (ESP32-S3 + integriertes SX1262-LoRa-Modul) als zentrale Rechen- und Funkeinheiten, eines im Flugzeug und eines in der Bodenstation
- Ein Arduino Nano (ATmega328) als separater, einfacher Servo-/Motor-Controller, angebunden über I2C an den Flugcontroller und über SBUS an den RC-Empfänger
- ESP-IDF (Version 5.x) als Firmware-Basis für Flugcontroller und Bodenstation, mit gemeinsam genutzten Komponenten in [shared_components/](#)
- Node.js/Vue 3 für die Weboberfläche, die als statische Dateien in den Flash-Speicher der Bodenstation eingebettet wird (kein separater Webserver)

4 Systemarchitektur

4.1 Hardwarearchitektur

4.1.1 Hauptkomponenten

Komponente	Einsatzort	Funktion
Heltec WiFi LoRa 32 V3 (ESP32-S3 + SX1262)	Flugzeug, Bodenstation (2×)	Hauptrechner + LoRa-Funkmodul
Arduino Nano (ATmega328)	Flugzeug	Servo-/Motoransteuerung, SBUS-Empfang, Sicherheits-Override
GPS-Modul (ATGM336H, NMEA)	Flugzeug (Bodenstation optional)	Positionsbestimmung
MPU6050 (Beschleunigungssensor + Gyroskop)	Flugzeug	Lagebestimmung (Roll/Pitch)
Magnetometer HMC5883L	Flugzeug	Kursbestimmung (Heading)
Barometer (BME280)	Flugzeug, Bodenstation	Höhenbestimmung über Luftdruck
SBUS-Empfänger	Flugzeug	Empfang der RC-Fernsteuerbefehle
4× Servo / ESC	Flugzeug	Querruder (differentiell), Höhenruder, Schub, Seitenruder
SD-/LittleFS-Speicher	Bodenstation	Ablage der Web-UI-Dateien im Flash
Akkus	Flugzeug, Bodenstation	Energieversorgung

Die vollständige Stückliste befindet sich in [BOM.md](#). Dort ist als Motor-/Servo-Controller ein Arduino Uno aufgeführt. Tatsächlich verbaut und in der Firmware adressiert ist jedoch ein Arduino Nano (siehe [motor_controller/platformio.ini](#)), eine kleine Inkonsistenz zwischen Beschaffungsliste und Aufbau.

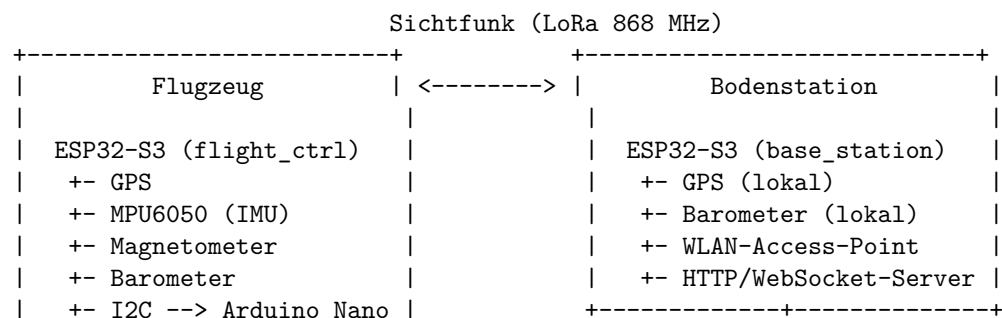
Beim Magnetometer wurde ursprünglich ein QMC5883P eingesetzt und im Projektverlauf durch ein HMC5883L ersetzt. Beide zugehörigen Datenblätter liegen unter [resources/](#) vor, der Grund für den Wechsel wird in Abschnitt „Technische Umsetzung und wesentliche Designentscheidungen“ erläutert.

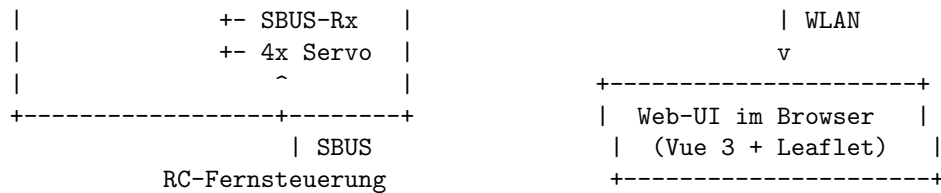
4.1.2 Mechanischer Aufbau

Unter [3d_models/](#) liegen die 3D-gedruckten Gehäuse-/Montagekonstruktionen:

- **Bodenstations-Gehäuse** ([base_station_case.scad](#)): zweiteiliges Gehäuse (Wanne + Deckel) mit Snap-Fit-Verbindung, Aufnahmen für die Hauptplatine (Heltec LoRa32 V3 + GPS-Modul) und eine über Kabel angebundene Tochterplatine (Barometer), einer SMA-Antennendurchführung in der Gehäusewand sowie einem Druckausgleichsloch im Deckel oberhalb des Barometers.
- **Flugzeug-Montageplatte** ([plane_mount_plate.scad](#)): laut Kommentar im Quellcode ein reines Layout-Mockup, das die relative Anordnung von Hauptplatine, zwei Tochterplatinen und den vier Steuerflächen-Servos zeigt. Es ist ausdrücklich kein flugtaugliches Bauteil (keine Rumpfbefestigung, keine Kabelkanäle, keine Gewichtsoptimierung).

4.1.3 Systemtopologie





Die Steuerflächen können sowohl vom Flugcontroller (über I2C) als auch direkt von der RC-Fernsteuerung (über SBUS) angesteuert werden. Welcher Pfad aktiv ist, entscheidet ausschließlich der Arduino im Flugzeug (siehe „Manuelle Übersteuerung“).

4.2 Softwarearchitektur

4.2.1 Projektstruktur

Pfad	Inhalt
<code>flight_controller/</code>	ESP-IDF-Firmware des Flugzeugs: Routenplanung, Sensorik, Flugregelung, LoRa-Kommunikation
<code>base_station/</code>	ESP-IDF-Firmware der Bodenstation: WLAN-AP, Web-UI-Server, LoRa-Relais
<code>motor_controller/</code>	Arduino-Firmware für Servo-/SBUS-Anbindung
<code>frontend_web/</code>	Vue-3-Weboberfläche der Bodenstation
<code>shared_components/</code>	Zwischen Flugcontroller und Bodenstation geteilte ESP-IDF-Komponenten (LoRa, I2C, Sensor-Treiber, Datenhaltung)
<code>python/</code>	Offline-Visualisierung der berechneten Flugrouten
<code>serial_plane_viz/</code>	Werkzeug zur Visualisierung der Servo-Ausschläge über die serielle Schnittstelle
<code>doc/, resources/</code>	Notizen, Pinouts, Datenblätter, Stückliste, Entwicklungsprotokoll

`flight_controller` und `base_station` sind zwei unabhängige ESP-IDF-Projekte mit jeweils eigener `sdkconfig/CMakeLists.txt`, die beide dieselben Komponenten aus `shared_components/` über `EXTRA_COMPONENT_DIRS` einbinden. Das ist die zentrale Wiederverwendungsstrategie des Projekts: Sensor-Treiber, I2C-Verwaltung, der LoRa-Funkstack und das Anwendungsprotokoll existieren nur einmal und werden von beiden Firmware-Projekten geteilt, gesteuert über ein projektweites Compile-Flag (`FLIGHT_DEVICE_TYPE_PLANE` bzw. `FLIGHT_DEVICE_TYPE_BASE_STATION`), das jeweils festlegt, welche Rolle ein Gerät im Protokoll einnimmt.

4.2.2 Kommunikationsschichten

Die Verbindung zwischen Flugzeug und Bodenstation ist in drei Schichten aufgeteilt:

- Funkschicht** (`shared_components/loracom`): Ansteuerung des SX1262-Funkmoduls über die RadioLib-Bibliothek. Da ein LoRa-Paket auf 255 Byte begrenzt ist, implementiert diese Schicht eine eigene Fragmentierung größerer Nachrichten (Kopf mit Nachrichten-ID, Fragment-ID und Gesamtfragmentzahl), eine Bestätigung jedes Fragments per ACK mit Zeitüberschreitung (2 s) und bis zu drei Wiederholungsversuchen, sowie periodische Keep-Alive-Pings (Standard: alle 30 s), um die Verbindung zu überwachen.
- Anwendungsprotokoll** (`shared_components/flightcom`): definiert typisierte Pakete auf Basis der Funkschicht, u. a. `SensorUpdate` (Luftdruck, Kurs), `PositionUpdate` (GPS-Position), `ComponentStatus` (Verbindungszustand der einzelnen Subsysteme, Override-Status, Flugzustand), `PlannedRoutePacket/PlannedAreaPacket` (berechnete Route bzw. vorgegebene Fläche) und `FlightHistoryPacket` (geflogene Strecke).
- Web-Anbindung** (Bodenstation → Browser): Die Bodenstation übersetzt die empfangenen LoRa-Pakete nicht direkt weiter, sondern schreibt sie in einen zentralen, thread-sicheren Zustandsspeicher (`FlightStorage`, Teil von `shared_components/flight_data`). Ein HTTP-/WebSocket-Server liest daraus und serialisiert die Daten getrennt als JSON für das Web-Frontend. LoRa-Wireformat und Web-JSON-Format sind damit vollständig entkoppelt.

4.2.3 Web-Stack der Bodenstation

Die Bodenstation hostet einen WLAN-Zugangspunkt (Standard-SSID, konfigurierbar über Kconfig, alternativ ein „Dev-Mode“, in dem sie sich stattdessen in ein bestehendes WLAN einbucht) sowie einen ESP-IDF-HTTP-Server. Dieser bedient:

- **Statische Dateien** der Weboberfläche aus einer LittleFS-Flash-Partition (`/*`-Route als Fallback-Handler). Das Verzeichnis `base_station/static` ist ein Symlink auf `frontend_web/dist`, sodass ein Frontend-Build direkt als Flash-Image eingebunden wird (`littlefs_create_partition_image` in `base_station/CMakeLists.txt`).
- **REST-Endpunkte**: u. a. `GET /api/status` (Verbindungsstatus), `POST/GET /api/area` (Zielfläche setzen/lesen), `GET /api/route` (berechnete Route abfragen).
- **WebSocket-Endpunkt** (`GET /api/ws`): sendet alle 5 Sekunden drei Nachrichtentypen an alle verbundenen Clients: `flight` (Positionen, geplante/geflogene Route), `connection` (Verbindungszustände aller Subsysteme), `sensor` (Luftdruck, berechnete Höhe, Kurs).

4.2.4 Frontend

Die Weboberfläche (`frontend_web/`) ist eine Vue-3-/TypeScript-Single-Page-Anwendung, gebaut mit Vite. Sie führt den Bediener über einen Assistenten (Zustandsautomat `Connection` → `Area Selection` → `Route Approval` → `Starting` → `Flying` → `Finishing`) durch die Missionsvorbereitung:

- **Verbindungsübersicht**: zeigt den Live-Status von Basis- und Flugzeugverbindung sowie deren Teilsysteme (GPS, Barometer, Motoransteuerung, Magnetometer) und gibt den nächsten Schritt erst frei, wenn alle Verbindungen stehen.
- **Flächenplanung**: Zeichnen eines Zielpolygons auf einer Leaflet-Karte (über `leaflet-draw`), Übertragung an die Bodenstation.
- **Routenvorschau**: Abfrage der vom Flugzeug berechneten Route, Darstellung auf der Karte zusammen mit der bisher geflogenen Strecke.
- **Live-Telemetrie**: laufende Aktualisierung von Position, Route und Sensorwerten über die WebSocket-Verbindung, ergänzt um einen eigenen Ping/Pong-Herzschlag im Frontend zur Erkennung von Verbindungsabbrüchen.

Für die Zustandsverwaltung nutzt die Anwendung einen selbst geschriebenen, reaktiven Store (`stores/store.ts`) auf Basis der Vue-3-`reactive()`-API. Pinia ist zwar als Abhängigkeit eingebunden und initialisiert, wird im Code aber nicht verwendet. Das ist ein kleiner, für die Funktion unschädlicher Altlast-Punkt im Projekt.

5 Technische Umsetzung und wesentliche Designentscheidungen

5.1 Routenplanung

Der `route_planner` erzeugt aus einem vom Bediener gezeichneten Polygon ein Mäander- bzw. Boustrophedon-Muster:

1. **Sweep-Lines erzeugen**: Aus der Bounding-Box des Polygons werden äquidistante, horizontale Linien (parallel zum Breitengrad) im Abstand der effektiven Schwadbreite berechnet.
2. **Zuschneiden auf das Polygon**: Jede Linie wird mit dem tatsächlichen, auch konkaven, Polygon geschnitten (Geometriebibliothek `homog2d`). Bei mehreren Schnittpunkten wird nur das äußere Segment behalten, sodass Löcher und konkave Formen korrekt ausgespart werden.
3. **Meridianbehandlung**: Ein Sprung über die 180°-Länge (Datumsgrenze) wird erkannt und die Längengrade vor den Geometrieoperationen in einen durchgängigen Wertebereich normalisiert.
4. **Pfad zusammensetzen**: Die einzelnen Linien werden abwechselnd von links nach rechts bzw. rechts nach links zu einem durchgehenden Pfad verbunden. Zwischen zwei Zeilen werden zusätzliche Zwischenpunkte eingefügt, um statt einer scharfen 180°-Kehre eine weichere Kurvenbahn zu erzeugen.
5. **Interpolation**: Abschließend werden alle Teilstrecken, die länger als ein konfigurierter Maximalabstand sind, in gleichmäßige Zwischenpunkte unterteilt, sodass zwei aufeinanderfolgende Wegpunkte nie weiter als dieser Abstand auseinanderliegen.

In der aktuellen Implementierung ruft der Flugcontroller die Routenplanung mit einer maximalen Punktdistanz von 60 m, einer Schwadbreite von 40 m und einem Überlappungsfaktor von 20 % auf, was einem effektiven Zeilenabstand von 32 m entspricht. Diese Werte sind im Quellcode fest hinterlegt statt über Kconfig konfigurierbar. Das ist eine bewusste Vereinfachung, da sie eng mit der Flughöhe und den Sensor-Auflösungen zusammenhängen.

5.2 Flugregelung

Der Flugcontroller (**FlightController**) läuft als FreeRTOS-Task mit einem festen Zyklus von 50 ms und implementiert eine kaskadierte PI-Regelung (bewusst ohne D-Anteil) für vier Steuergrößen:

- **Seitenruder (Kurs):** Aus der aktuellen GPS-Position und dem nächsten Wegpunkt wird die Soll-Peilung (Bearing) berechnet und mit dem gemessenen Kompasskurs verglichen. Der normalisierte Peilungsfehler wird proportional (P-Regelung) auf das Seitenruder abgebildet.
- **Höhenhaltung:** Die Differenz aus Zielhöhe (60 m) und gemessener Höhe treibt zwei parallele PI-Regelungen: eine für den Schub und eine für den Ziel-Pitch-Winkel, der wiederum als Sollwert in die Nickregelung einfließt (kaskadierte Regelung).
- **Querruder:** Reine „Wings-Level“-Regelung (PI auf den Rollwinkel), unabhängig vom Kurvenkommando. Es gibt aktuell keine koordinierte Kurve, das Abbiegen erfolgt ausschließlich über das Seitenruder.
- **Höhenruder:** PI-Regelung des Nickwinkels auf den von der Höhenregelung vorgegebenen Ziel-Pitch.

Die Wegpunkt-Erreichung wird über den Abstand zur aktuellen Position gegen einen konfigurierbaren Radius (Standard 30 m) geprüft. Ist der letzte Wegpunkt erreicht, wechselt der interne Flugzustand auf „Rückkehr“. Reglerverstärkungen und Zielhöhe sind, anders als zum Beispiel Telemetrieintervall oder Wegpunktradius, als Konstanten im Quellcode hinterlegt statt als Kconfig-Parameter. Das spiegelt eine bewusste Priorisierung von Einfachheit gegenüber Laufzeit-Konfigurierbarkeit während der Entwicklungsphase wider.

5.3 Sensorik und Datenfusion

Die Sensordatenverarbeitung ist bewusst einfach gehalten: Es kommt **kein** gemeinsamer Zustandsschätzer (z. B. Kalman-Filter) zum Einsatz. Stattdessen wird jeder abgeleitete Messwert unabhängig über ein gleitendes Mittel geglättet und direkt in die jeweilige PI-Regelung eingespeist:

- **Lage (Roll/Pitch):** wird ausschließlich aus der Schwerkrafttrichtung des Beschleunigungssensors berechnet (**atan2** über die Beschleunigungskomponenten), nach einer Nullpunktkalibrierung beim Start. Ein echter Komplementärfilter (Fusion aus Gyroskop- und Beschleunigungsdaten) sowie eine reine Gyroskop-Integration für die Gierrate sind im Quellcode vorhanden, werden in der aktuellen Regelschleife jedoch nicht verwendet. Sie sind damit vorbereitet, aber (noch) nicht aktiviert.
- **Kurs (Heading):** wird direkt aus den Magnetometer-Rohwerten berechnet (**atan2**), ohne Neigungskompensation gegenüber Roll/Pitch und ohne Korrektur der magnetischen Missweisung (Deklination). Anschließend wird der Wert zirkulär geglättet (getrennte Mittelung von Sinus-/Kosinuskomponenten, um den Sprung bei 0°/360° korrekt zu behandeln).
- **Höhe:** aus dem barometrischen Luftdruck über die hydrostatische Grundgleichung, bezogen auf einen von der Bodenstation übermittelten Referenzdruck am Boden. Die berechnete Höhe ist somit relativ zum Startpunkt (AGL) und nicht absolut (MSL).
- **Position:** direkt aus geparsten GPS-NMEA-Sätzen, ohne zusätzliche Filterung oder Koppelnavigation. Eine Position gilt erst ab einer Mindestanzahl empfangener Satelliten als gültig.

Diese Architektur ist als pragmatischer Zwischenstand zu verstehen. Sie liefert brauchbare, aber gegenüber Vibrationen und kurzfristigen Störungen empfindlichere Schätzwerte als eine echte Sensorfusion (siehe „Bekannte Einschränkungen“).

5.4 Kommunikation Flugcontroller / Motor-Controller

Der Arduino-Motor-Controller ist I2C-Slave (Adresse 0x42) mit einem bewusst minimalen, registerlosen Protokoll:

- **Schreiben (Flugcontroller → Arduino):** genau 4 Byte, je ein vorzeichenbehafteter Wert im Bereich -100...100 für Seitenruder, Höhenruder, Schub und differentiell Querruder. Unvollständige Übertragungen werden verworfen, die zuletzt gültigen Werte bleiben erhalten.
- **Lesen (Arduino → Flugcontroller):** ein einzelnes Statusbyte, das anzeigt, ob der manuelle Override aktiv ist. Ein erfolgreicher Lesevorgang dient gleichzeitig als Verbindungsprüfung.

Der Arduino überwacht selbstständig die Aktualität eingehender I2C-Nachrichten und initialisiert den I2C-Bus automatisch neu, wenn eine Sekunde lang keine Nachricht eintrifft. Das ist eine Absicherung gegen hängende Bus-Zustände, die im Projektverlauf mehrfach beobachtet wurden.

5.5 Manuelle Übersteuerung

Die sicherheitskritische Entscheidung, ob das Flugzeug autonom oder manuell gesteuert wird, ist bewusst **nicht** im Flugcontroller, sondern lokal auf dem Arduino getroffen: Ein festgelegter SBUS-Kanal des RC-Empfängers wird als Override-Schalter interpretiert. Überschreitet sein Wert eine Schwelle, steuert der Arduino die vier Servos direkt aus den SBUS-Kanälen an und ignoriert die zuletzt vom Flugcontroller empfangenen I2C-Werte. Der Flugcontroller selbst liest den Override-Status nur zu Anzeige-/Telemetriezwecken mit, hat aber keinen Einfluss darauf. Diese Entkopplung stellt sicher, dass die manuelle Kontrolle auch bei einem Absturz oder Hänger der ESP32-Firmware erhalten bleibt.

5.6 Wesentliche Designentscheidungen im Projektverlauf

- **Wechsel von PlatformIO zu ESP-IDF** für Flugcontroller und Bodenstation (Woche 4 des Entwicklungsprotokolls): Auslöser waren Schwierigkeiten in der Komponentenverwaltung unter PlatformIO. ESP-IDF ermöglichte zudem, dieselben Komponenten nativ auf dem Host-Rechner zu bauen und zu testen (siehe „Tests und Evaluation“).
- **Wechsel des Magnetometers von QMC5883P auf HMC5883L** sowie Umstellung des Auslesemodus von „Continuous“ auf „Single-Read“ (Wochen 10 und 11): Das ursprünglich verbaute QMC5883P antwortete unzuverlässig und teils unter wechselnden I2C-Adressen. Die tiefere Ursache eines wiederkehrenden Bus-Ausfalls wurde später identifiziert: Im kontinuierlichen Messmodus kann ein Lesezugriff während einer laufenden internen Messung den Sensor in einen Zustand versetzen, der den gesamten I2C-Bus blockiert bzw. mit Stördaten belegt, statt nur einen einzelnen fehlerhaften Messwert zu liefern. Die Umstellung auf einzeln angeforderte Messungen (Single-Read) behebt dieses Verhalten, da der Sensor Lesezugriffe erst nach Abschluss einer Messung zulässt.
- **Eigenes Fragmentierungsprotokoll über LoRa**: Da einzelne Anwendungsnachrichten (z. B. eine vollständige geplante Route) die maximale LoRa-Paketgröße von 255 Byte überschreiten können, wurde ein eigenes Fragmentierungs- und Bestätigungsschema mit Nachrichten-/Fragment-IDs entwickelt.
- **Trennung von Funkprotokoll und Web-Protokoll**: Die bewusste Entkopplung von LoRa-Wireformat und WebSocket-/REST-JSON-Format über den zentralen `FlightStorage`-Zustandsspeicher erlaubt es, beide Seiten unabhängig voneinander weiterzuentwickeln.

5.7 Begleitende Werkzeuge

Zwei Werkzeuge unterstützen die Entwicklung, sind aber nicht Teil der eigentlichen Flugsoftware:

- **python/**: Zwei kleine Skripte zur Offline-Visualisierung der vom Routenplaner erzeugten Sweep-Pfade, als einfacher 2D-Plot (`matplotlib`) oder auf einer interaktiven Karte (`folium`).
- **serial_plane_viz/**: Ein Browser-Werkzeug (`three.js`), das Servo-Ausschläge (Motor, Roll, Pitch, Gier), die über eine serielle Schnittstelle im selben Wertebereich wie das I2C-Protokoll des Motor-Controllers gesendet werden, an einem einfachen 3D-Flugzeugmodell visualisiert. Das ist nützlich zur Fehlersuche ohne reales Flugzeug.

6 Tests und Evaluation

6.1 Automatisierte Tests

Flugcontroller (native Unit-Tests): Das Build-System von `flight_controller` unterscheidet anhand einer Umgebungsvariable zwischen einem echten ESP-IDF-Firmware-Build und einem nativen Host-Build. Für Letzteren existiert ein eigener CMake-Kompatibilitäts-Layer, der ESP-IDF-Komponenten als gewöhnliche CMake-Bibliotheken für den Host kompilierbar macht, sodass die Geschäftslogik ohne reale Hardware getestet werden kann (GoogleTest). Inhaltlich beschränken sich die vorhandenen Tests ausschließlich auf die **Geometrie der Routenplanung**: korrekte Erkennung der äußeren Polygonpunkte, korrekte Schnittpunktberechnung zwischen Linie und Polygon, korrekte Anzahl/Lage der erzeugten Sweep-Lines sowie ein monotoner Verlauf des zusammengesetzten Sweep-Pfads. Es existieren **keine** automatisierten Tests für die Flugregelung (PI-Regler), die Sensor-Treiber oder das I2C-Protokoll zum Motor-Controller. Dieser Teil wird ausschließlich auf echter Hardware verifiziert.

Frontend: Typprüfung (`vue-tsc`) und Linting (ESLint/oxlint) laufen als Basis-Qualitätssicherung bei jedem Build. Ein Playwright-E2E-Grundgerüst ist eingerichtet, der einzige vorhandene Test ist jedoch der unveränderte Gerüst-Test der Vue-Projektvorlage, der auf einen Beispieltext prüft, den die tatsächliche

Anwendung nicht mehr enthält. Dieser Test deckt keine reale Funktionalität ab und ist als offene Lücke zu betrachten, nicht als funktionierende Testabdeckung.

Kontinuierliche Integration: Die GitLab-CI-Pipeline ([.gitlab-ci.yml](#)) erzeugt ausschließlich die PDF-Fassungen dieser Dokumentation per Pandoc (`make readme_pdf`). Sie baut weder die ESP-IDF-Firmware noch die Arduino-Firmware noch das Frontend, und sie führt auch die vorhandenen GoogleTest-Unit-Tests nicht automatisiert aus. Bauen und Testen der Firmware-Komponenten (`idf.py build, pio run, ctest`) sind aktuell manuelle, lokale Schritte. Das ist eine Lücke in der Qualitätssicherung des Projekts.

6.2 Evaluation anhand des Entwicklungsverlaufs

Das Entwicklungsprotokoll ([resources/learning.md](#)) dokumentiert den Fortschritt über zwölf Wochen und erlaubt eine ehrliche Einschätzung des tatsächlich erreichten Funktionsstands:

- Erfolgreich umgesetzt und in Betrieb genommen wurden: GPS-Auswertung, eine funktionierende LoRa-Verbindung mit Bestätigungen und Keep-Alive-Pings, die I2C-Anbindung von Barometer und IMU sowie, nach den beschriebenen Hardware-Problemen, eine funktionsfähige Magnetometer-Anbindung.
- Mehrere grundlegende technische Probleme mussten während der Entwicklung gelöst werden, u. a. eine notwendige Paketfragmentierung für LoRa-Nachrichten über 255 Byte, ein Deadlock in der Event-Handler-Warteschlange sowie die oben beschriebenen I2C-Bus-Aussetzer im Zusammenhang mit dem Magnetometer.
- Laut dem Abschnitt „Outlook“ des Protokolls waren zum Zeitpunkt des letzten Eintrags die **Rollregelung im Horizontalflug, die kombinierte Höhenhaltung, eine automatische Schubregelung sowie eine autonome Landung noch nicht umgesetzt bzw. nicht flug-erprobt**. Der Regelkreis existiert im Code (siehe oben), wurde aber im dokumentierten Zeitraum nicht vollständig im realen Flug verifiziert. Die Evaluation des Gesamtsystems ist entsprechend als „Boden- und Teilsystem-erprobt, noch nicht vollständig flugerprobt“ einzustufen.
- Wiederkehrende I2C-Bus-Instabilität beim gleichzeitigen Betrieb mehrerer Sensoren am selben Bus war das am längsten offene Hardware-/Firmware-Problem im Projektverlauf und ist auch bei den implementierten Workarounds (Bus-Recovery, Single-Read-Modus) als grundsätzliches Restrisiko zu betrachten.

7 Bekannte Einschränkungen und mögliche Verbesserungen

Bereich	Einschränkung	Mögliche Verbesserung
Sensorfusion	Lagebestimmung rein aus Beschleunigungssensor, kein Kalman-/Komplementärfilter aktiv, Kompass ohne Neigungskompensation/Deklinationsskorrektur	Vorhandenen Komplementärfilter und Gyroskop-Integration tatsächlich in die Regelschleife einbinden, Neigungskompensation und Deklination im Heading ergänzen
Regelung	Reglerverstärkungen und Zielhöhe hart codiert statt konfigurierbar, keine koordinierte Kurve (Querruder unabhängig vom Kurvenkommando), volle Regelkaskade nicht flugerprobt	Gains über Kconfig/Laufzeit konfigurierbar machen, systematische Flugtests durchführen, koordinierte Kurvenregelung ergänzen
Sicherheit	Ausgelieferte Motor-Controller-Firmware ist mit Debug-Defines gebaut, die den SBUS-Override-Sicherheitsschalter deaktivieren	Debug-Defines vor jedem realen Flugbetrieb zwingend entfernen, ggf. als eigene Build-Variante trennen, um Fehlbedienung auszuschließen
Hardware-Robustheit	Wiederkehrende I2C-Bus-Aussetzer bei mehreren angeschlossenen Sensoren, trotz Workarounds ein Restrisiko	Bus-Topologie überdenken (z. B. I2C-Multiplexer, getrennte Busse), Hardware-Redesign der Verkabelung

Bereich	Einschränkung	Mögliche Verbesserung
Funkstrecke	Keine Verschlüsselung aktiv (AES-128-GCM im Code vorbereitet, aber nicht verdrahtet), keine Geräteadressierung (nur Punkt-zu-Punkt)	Verschlüsselung aktivieren, Adressierung für Mehrflugzeug-Szenarien ergänzen
Testabdeckung	Nur die Routenplanungs-Geometrie ist automatisiert getestet, CI baut ausschließlich die Dokumentation und nicht die Firmware/das Frontend, der Frontend-E2E-Test ist unveränderter Gerüst-Code	Unit-Tests für Regelungslogik und Protokollcode ergänzen, Firmware-/Frontend-Build und Tests in die CI-Pipeline aufnehmen, Playwright-Test an die reale Anwendung anpassen
Frontend-Code	Pinia eingebunden, aber ungenutzt, MapView -Komponente nutzt einen anderen Leaflet-Ansatz als der Rest der App und ist nicht in den eigentlichen Bedien-Workflow eingebunden	Aufräumen: Pinia entfernen oder konsequent nutzen, MapView entfernen oder konsolidieren
Funktionsumfang	Kamera-Nutzlast, automatisierte Landung (z. B. per Laser-Entfernungsmessung) und Bildübertragung aus der ursprünglichen Projektidee sind nicht umgesetzt	Als nächste Ausbaustufen nach Abschluss der Kernnavigation angehen
Dokumentation	Stückliste nennt einen Arduino Uno, tatsächlich verbaut ist ein Arduino Nano	Stückliste an den tatsächlichen Aufbau anpassen

8 Individuelle Beiträge

Das Projekt wurde als Einzelarbeit umgesetzt (ein Autor laut Aufgabenstellung und Versionskontrolle). Konzeption, die gesamte Firmware-Entwicklung (Flugcontroller, Bodenstation, Motor-Controller), die Entwicklung der Weboberfläche, die Auswahl und Integration der Hardware sowie diese Dokumentation stammen vollständig von derselben Person. Eine Aufteilung auf mehrere Bearbeiter entfällt entsprechend.